

How to Debug a Multi-Threaded Application in C#

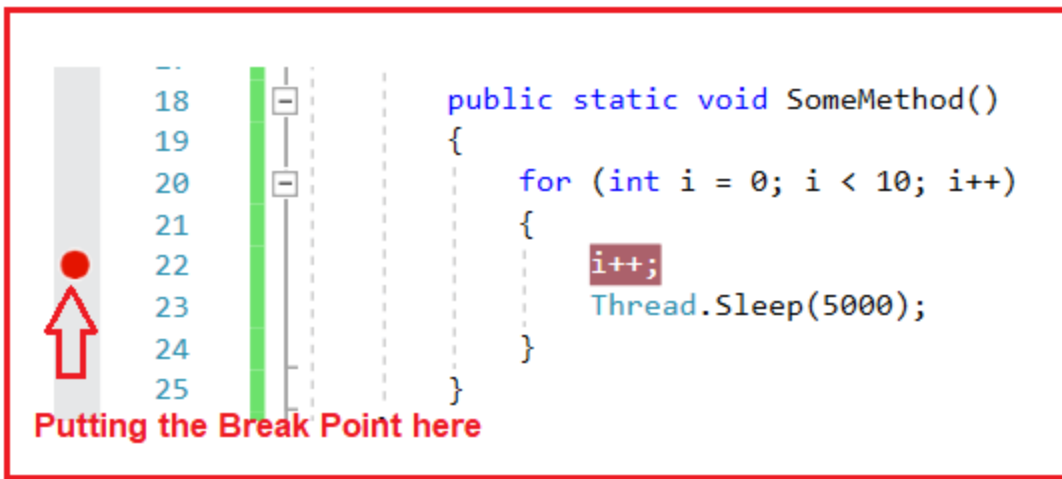
In this article, I am going to discuss **How to Debug a Multi-Threaded Application in C#** with Examples. Please read our previous article where we discussed [Inter Thread Communication in C#](#) with Examples.

How to Debug a Multi-threaded Application in C#?

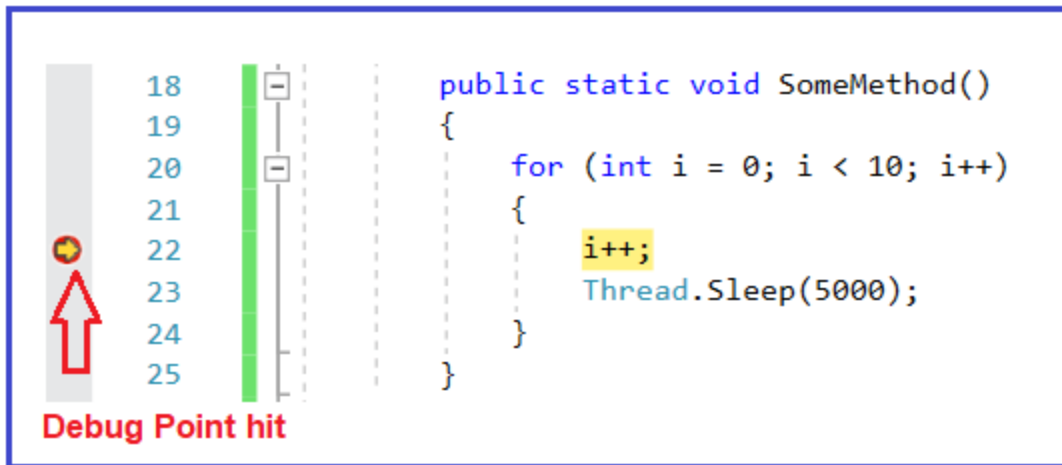
Let us understand how to debug threads in C# using Visual Studio. Please have a look at the below example. In the below example, we have a method called SomeMethod and this SomeMethod contains a for loop which will run 10 times. As part of the method body, it just manipulates the i variable and then sleeps for 5 seconds. Then we invoke the SomeMethod from the Main method by creating two different threads i.e. thread1 and thread2 and then call the Start method.

```
using System;
using System.Threading;
namespace DebugThreadsDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            Thread thread1 = new Thread(SomeMethod);
            Thread thread2 = new Thread(SomeMethod);
            thread1.Start();
            thread2.Start();
            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            for (int i = 0; i < 10; i++)
            {
                i++;
                Thread.Sleep(5000);
            }
        }
    }
}
```

Now, we want to debug the for loop of the SomeMethod of the above Console application. So, normally what we are intended to do is, we need to put the breakpoint inside the for loop as shown in the below image.

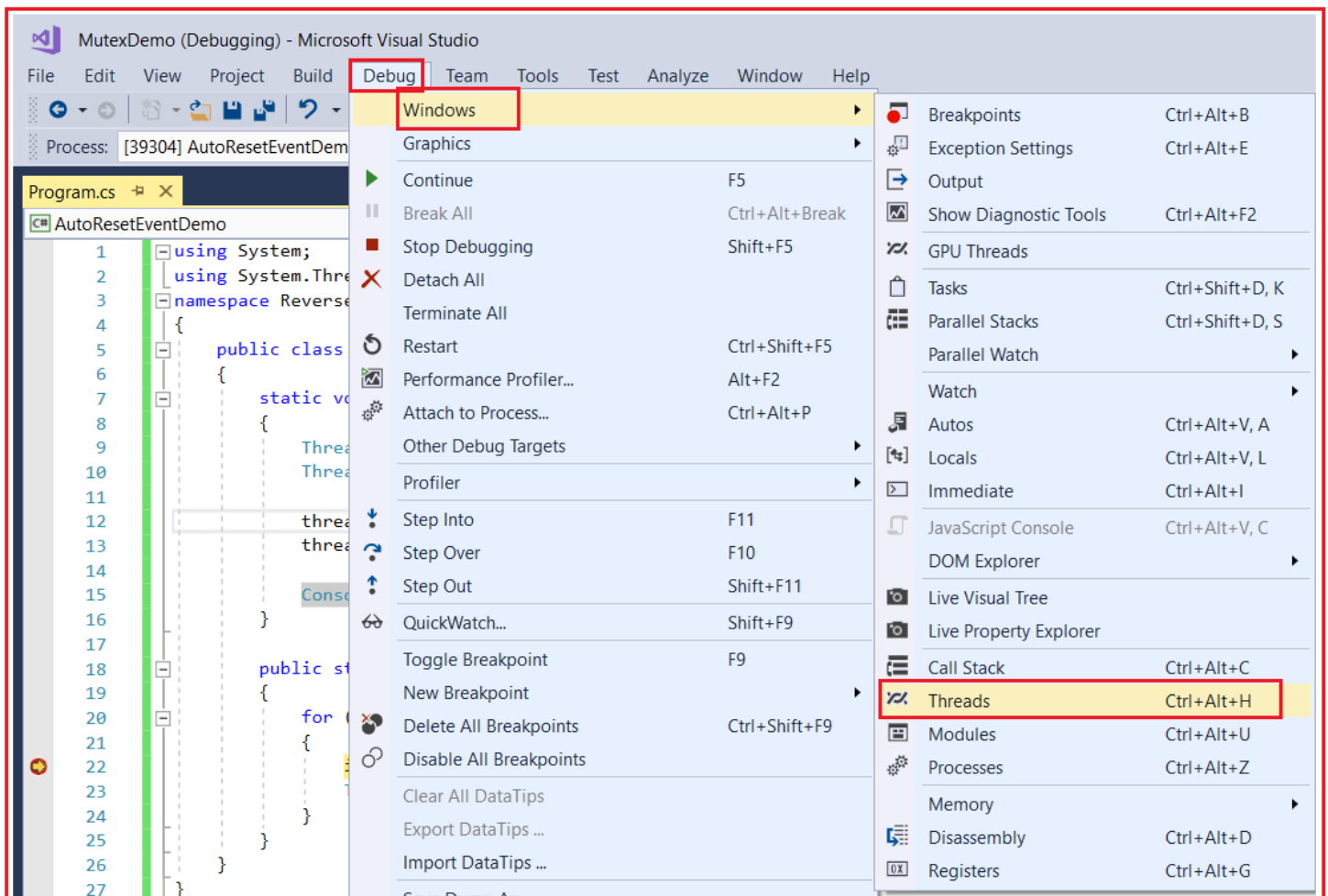


Once you put the debug point, run the application. Once you run the application it will hit the debug point as shown in the below image.



But this debugging has problems. The first problem with this debugging approach is that I am unable to identify currently my debugger is debugging for which thread, whether it is debugging thread1 or thread2, I am unable to identify.

In order to identify the debugger is debugging which thread, just select **Debug => Windows => Threads** options from the context menu as shown in the below image.



So, once you select the **Debug => Windows => Threads** options, it will open the following window. The yellow symbol shows where the current debugger is debugging. In the Location header, you can see the name of the method and from there you can identify the threads. Here, you can see in the Location header, that it is showing three method names i.e. Main and two times SomeMethod. The Main thread executes the Main method and the Worker Threads execute the SomeMethod.

	ID	Managed ID	Category	Name	Location
^ Process ID: 34924 (3 threads)					
▼	33024	1	Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	31096	3	Worker Thread	<No Name>	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	22888	4	Worker Thread	<No Name>	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod


This Worker Thread is Currently Debugging

You can see in the above image that the name for worker threads is showing **<No Name>**. For a better understanding, let us give some meaningful names to our threads. So, modify the example code as follows. Here, we have provided the thread1 name as Thread One and the thread2 name as Thread Two.

```
using System;
using System.Threading;
namespace DebugThreadsDemo
{
    public class Program
```

```

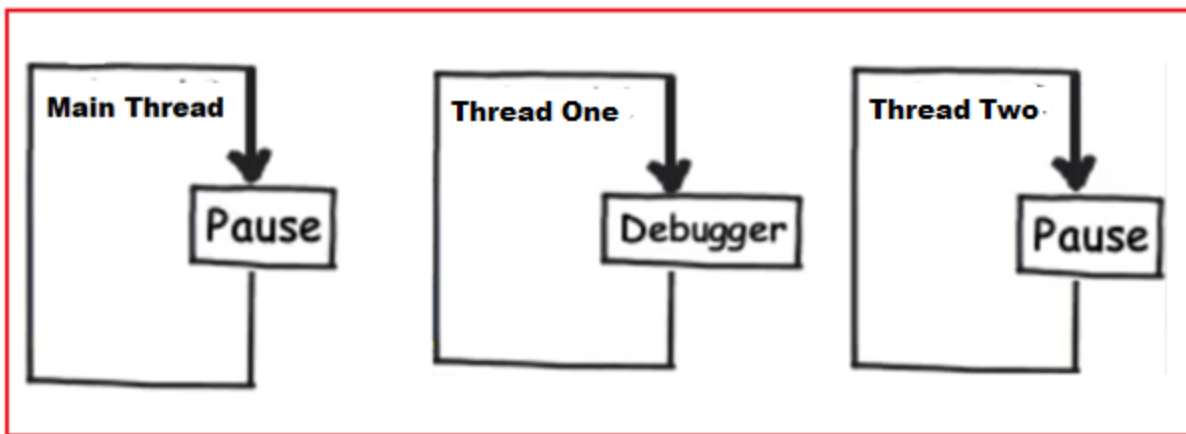
{
    static void Main(string[] args)
    {
        Thread thread1 = new Thread(SomeMethod)
        {
            Name = "Thread One"
        };
        Thread thread2 = new Thread(SomeMethod)
        {
            Name = "Thread Two"
        };
        thread1.Start();
        thread2.Start();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        for (int i = 0; i < 10; i++)
        {
            i++;
            Thread.Sleep(5000);
        }
    }
}
}

```

Now, save the changes. Put the debugger point and run the application. Once you run the application and once the debugger point is hit, open **Debug => Windows => Threads**, and this time it will show you the name of the worker's thread as shown in the below image. Now, the yellow symbol is on Thread two which means it is debugging Thread Two currently. In your case, it might be pointed to Thread One. If you see only one thread, then simply click on F5, and then you will see both threads.

	ID	Managed ID	Category	Name	Location
^ Process ID: 33208 (3 threads)					
▼	35648	1	✖ Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	21912	3	⚙ Worker Thread	Thread One	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	29888	4	⚙ Worker Thread	Thread Two	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod

When our debugger is debugging a thread, then all other threads including the main thread go into the halt mode or you can say pause mode. Suppose, our debugger is debugging Thread One, then don't think that Thread two is running parallelly. As the debugger halted at the SomeMethod i.e. our application code, then all other threads including the main thread will be halted. For a better understanding, please have a look at the below diagram.



As you can see in the **Debug => Windows => Threads** window, currently thread two is debugging. If you want to switch i.e. you want to debug Thread One, then simply select Thread One and double-click on it as shown in the below image.

	ID	Managed ID	Category	Name	Location
^ Process ID: 33208 (3 threads)					
▼	35648	1	Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	21912	3	Worker Thread	Thread One	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	29888	4	Worker Thread	Thread Two	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod

First Select this one and then double click on it to Debugging Thread One

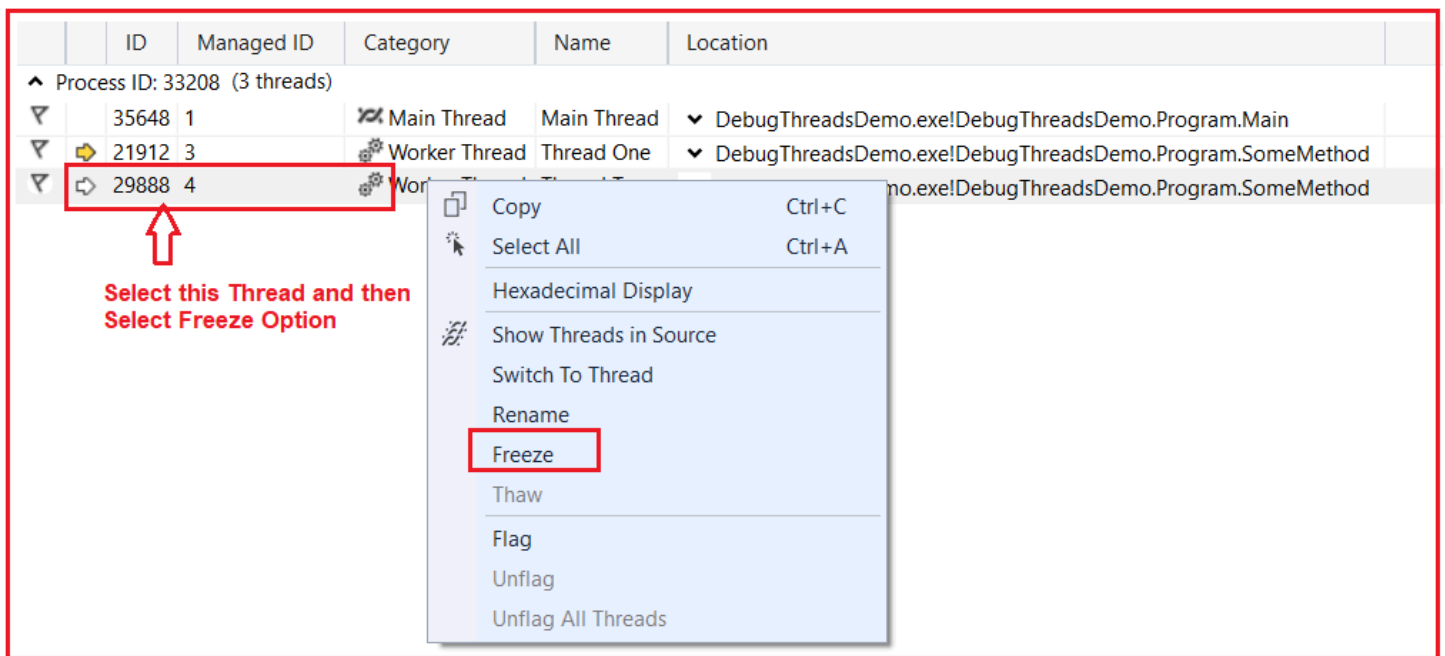
Once you select and double-click on Thread One, then you will see that the yellow symbol is switched to Thread One as shown in the below image which means currently it is debugging Thread One.

	ID	Managed ID	Category	Name	Location
^ Process ID: 33208 (3 threads)					
▼	35648	1	Main Thread	Main Thread	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.Main
▼	21912	3	Worker Thread	Thread One	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod
▼	29888	4	Worker Thread	Thread Two	▼ DebugThreadsDemo.exe!DebugThreadsDemo.Program.SomeMethod

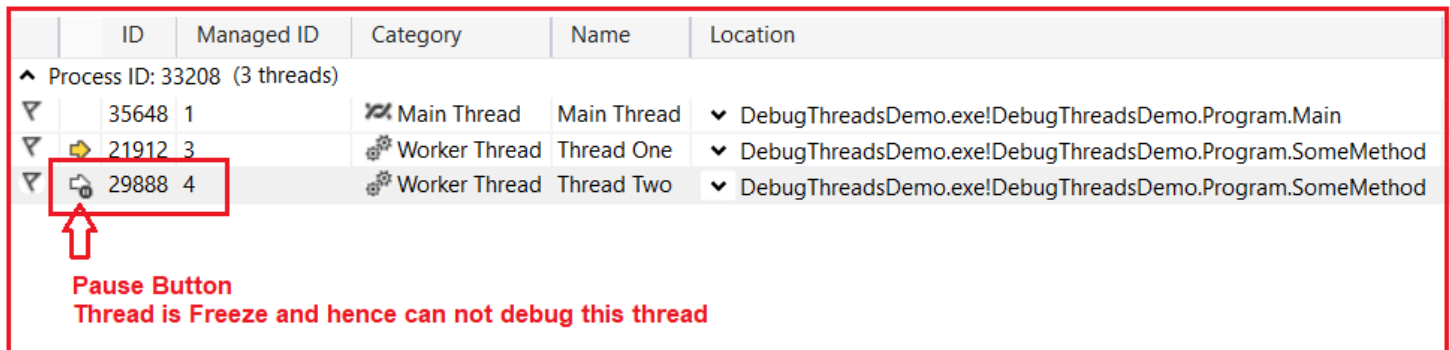
How to Debug a Specific Thread in C#?

Suppose you want to debug only Thread One. You don't want to debug Thread Two. This is also possible. Visual Studio provides two options i.e. **Freeze** and **Thaw**. So, when you select the Freeze option of a thread, then that thread is not going to be debugged by a debugger. Similarly, if you select the Thaw option of a thread, then again it is going to be debugged by the debugger. When a thread is created in C#, by default it is created with the Thaw option.

Now, let us see how to pause Thread Two so that our debugger only debugs Thread One. To do so, right-click on Thread One and then select the Freeze option from the context menu as shown in the below image.

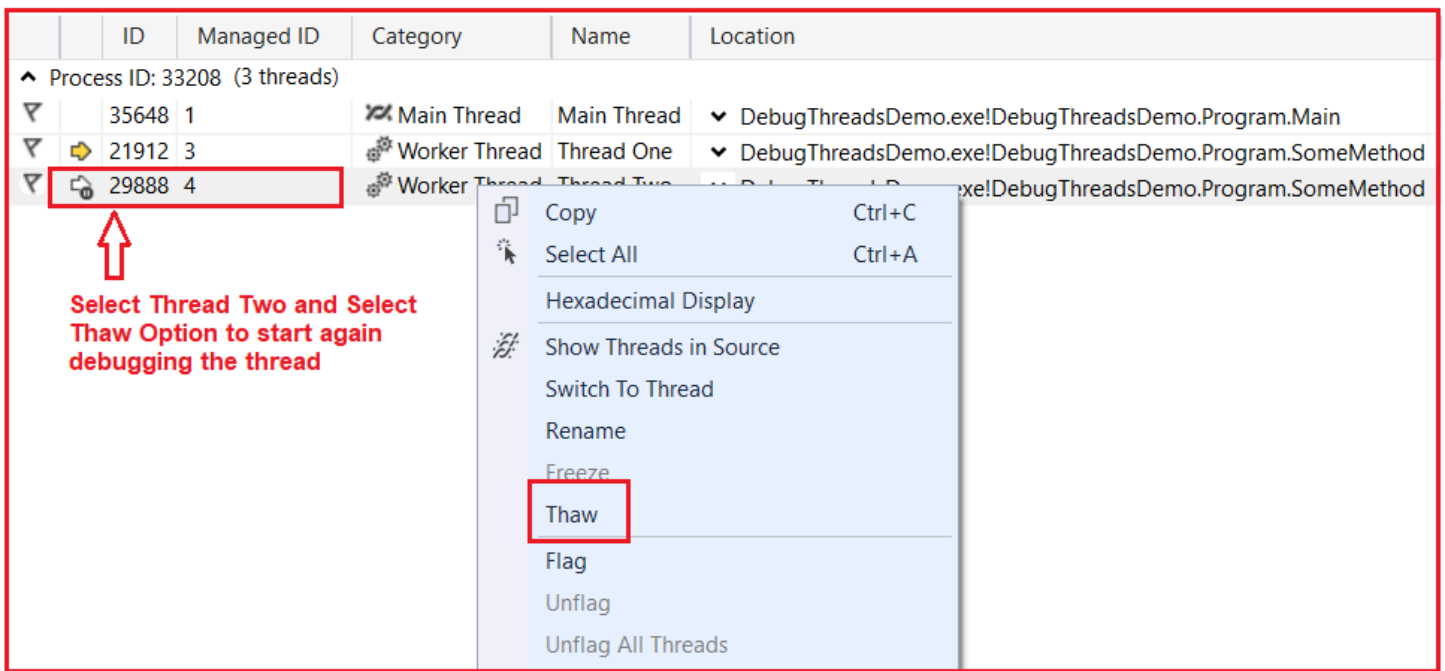


Once you select the Thread and click on the Freeze, then it will Freeze the thread and you can also see the Pause button as shown in the below image.



With the above changes in place, now if you continue your debugging, then you will see that it will only Debug Thread One, it will not debug Thread Two.

Now, again if you want to debug Thread Two, then simply you need to select Thread two, and this time you need to select the Thaw option from the context menu as shown in the below image.

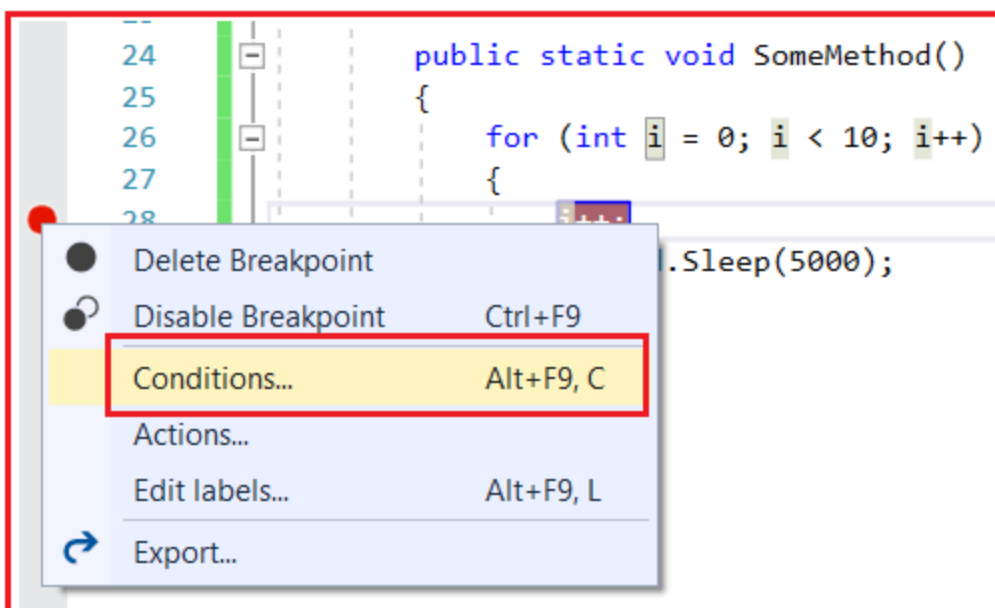


Note: You can use the Freeze option to stop debugging and the Thaw option to allow debugging.

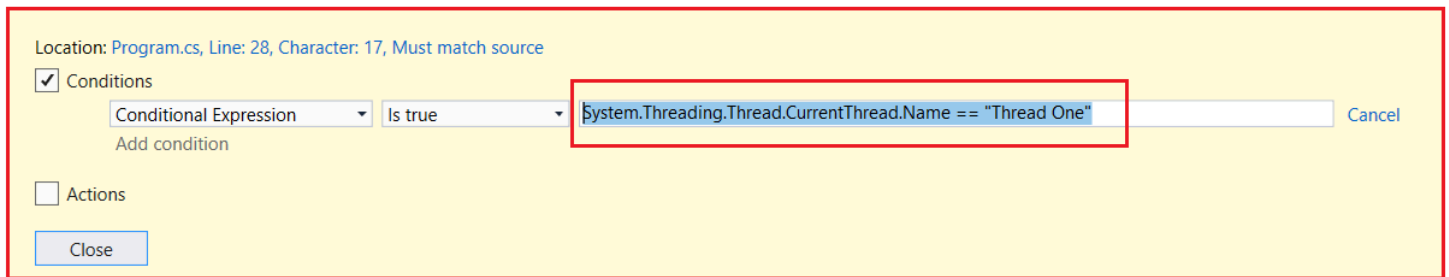
What is Debugger Point with Condition in C#?

Sometimes you want to debug specifics. Let us understand this in detail. In our example, when we run the application in debug mode, sometime it will halt Thread Two if Thread One is debugging or it will halt Thread One if Thread Two is debugging. But what about if you want to only halt the debugger on Thread One, I am not interested to halt the debugger on Thread two. If you want to do so, then you need to create the debugger point with a condition. Let us see the procedure of how to create a debugger point with the condition.

First, right-click on the debugger point and then select the Conditions option from the context menu as shown in the below image.



Once you click on the Conditions option, then it will open the below window. In conditions, we are writing **System.Threading.Thread.CurrentThread.Name == "Thread One"** which means it will only hit the debug point if the thread is Thread One.



Location: Program.cs, Line: 28, Character: 17, Must match source

☒ Conditions

Conditional Expression Is true System.Threading.Thread.CurrentThread.Name == "Thread One" Cancel

Add condition

☐ Actions

Close

With the above changes in place, now if you run the application, then the debugger point will halt only for Thread One. So, by giving a name to our thread how easier it is for us to debug a multithread application in C#.

In the next article, I am going to discuss [Array in C#](#) with Examples. Here, in this article, I try to explain **How to Debug a Multi-threaded Application in C#** with Examples. I hope you enjoy this How to Debug a Multi-threaded Application in C# with Examples article.